Neighbourhood-Aware V2X Misbehaviour Detection with Plausibility Checks and a Collusion Stress Test

Shlok (UAI05BTA2317), Jiya (UAI05BTA2313), Caleb (UAI05BTA2319)
BTech AI ML, Universal AI University

Abstract—Connected vehicles broadcast Basic Safety Messages (BSMs) that state their position, speed and heading several times per second, and every receiving vehicle builds its picture of the road from them. A vehicle that broadcasts false content (a shifted or frozen position, a wrong speed, replayed or delayed messages, or a flood of messages) can make its neighbours brake, change lanes or ignore a real hazard. Digital signatures do not help here: they prove who sent a message, not that its content is true, so the content must be judged on the receiving vehicle itself. Current learned detectors leave four gaps. They judge each sender on its own, ignoring the other vehicles the receiver hears at the same time; they are evaluated on few attack types and often report accuracy alone, which hides the false-alarm rate; they label every message of an attacker vehicle as malicious even while it behaves; and they are rarely tested against attackers that tell a physically consistent story or that collude. We study the receiver's view. For every receiving vehicle and second, the 16 nearest senders over the last 10 seconds are classified as honest or misbehaving by a small transformer (144k parameters) that attends over each sender's history and across the neighbouring senders. Its inputs add plausibility checks to the raw claims: kinematic consistency, a road map learned from honest traffic, exact repeats of earlier positions and positions copied from another sender's trail. Training adds windows in which honest senders tell careful, mutually consistent lies, and a receiver-side memory keeps suspicion of senders already flagged. The system is implemented in PyTorch and trains in 10–15 minutes on a laptop GPU. On the public VeReMi Extension benchmark (simulated, 14 of its 19 attack types, 68,971 test windows, two seeds) it reaches precision 98.16%, recall 97.78%, F1 97.97% and accuracy 98.74% when an attacker is scored only in windows where it sends falsified messages, and F1 97.01% when every window of an attacker vehicle counts, against F1 94.9% for the same model without the checks and 78.9% for the dataset's speed-consistency rule under that stricter labelling. Neighbour attention adds a small but significant 0.3 F1 points and lowers the false-positive rate from 0.90% to 0.60%, but its benefit disappears when most neighbours are attackers. A collusion stress test shows why the checks matter: before them, coordinated colluders with consistent lies were caught in at most 4% of cases; with them, in 70–90% at a 1% false-positive rate, including a spoofed convoy never seen in training. Lies that stay on the road and obey the physics remain hard (23–48%), which marks the limit of detection from message content alone.

Index Terms—V2X, VANET, misbehaviour detection, transformer, plausibility checks, collusion, VeReMi Extension

I. The Problem

Setting. In V2X networks every vehicle periodically broadcasts a BSM with its claimed position, speed, heading and acceleration. Message signatures prove who sent a message but not that its content is true: an authenticated insider, or a vehicle with a faulty or manipulated sensor, can send false content that passes all cryptographic checks [11], [18]. Misbehaviour detection must therefore judge the content itself, on the receiving vehicle, within a fraction of a second, without ground truth about where the other vehicles really are.

Task. Let receiver r at second t hear a set of senders. We take the K=16 senders closest to r by claimed distance and their claims over the last T=10 seconds (messages binned to one-second steps) and output, for each sender, the probability that it misbehaves. A window is formed every 3 seconds; each sender slot is one decision. The receiver knows its own position but never the true position of a sender.

Threat model. The attacker is an insider: a vehicle with valid credentials that controls the content and the timing of its own messages. It may lie about its position, speed, heading and acceleration, replay or delay messages, copy other vehicles' content, or flood the channel. Several attackers may coordinate (collusion). The attacker cannot change what the receiver measures about itself, and it does not control the honest vehicles. The detector runs on the receiver and sees only received messages and the receiver's own state; it has no access to roadside units, other receivers' decisions or the physical layer.

Why it is hard. Honest claims are themselves noisy (GPS error of a few metres, sensor error on speed and heading), vehicles stop and accelerate legitimately, and a receiver hears a changing set of senders whose messages arrive irregularly. The detector must keep false alarms below about 1%, because every false alarm discards an honest vehicle's data, and must decide within seconds. A lie that is small, smooth and on the road can look exactly like an honest vehicle, and when several attackers agree, the neighbours a receiver could compare against are partly malicious themselves.

Attacks. We use VeReMi Extension [2], the successor of VeReMi [1]: simulated traffic of the Luxembourg scenario with 30% misbehaving vehicles and 19 attack types. We run 14 of them: constant, random and offset positions (ConstPos, RandomPos, ConstPosOffset, RandomPosOffset), constant, random and offset speeds (ConstSpeed, RandomSpeed, ConstSpeedOffset), EventualStop (the vehicle claims to have stopped), Disruptive and DataReplay (content taken from other vehicles), DelayedMessages, and three denial-of-service variants (DoS, DoSRandom,

TABLE I
The 14 attack types used, what the attacker changes, and the number of attacker slots in the test set (message labels).

Attack	What the attacker sends	Test slots
ConstPos	a fixed position	6,179
ConstPosOffset	true position plus a fixed offset	6,188
RandomPos	a random position each message	6,179
RandomPosOffset	true position plus a random offset	6,189
ConstSpeed	a fixed speed	6,158
ConstSpeedOffset	true speed plus a fixed offset	6,153
RandomSpeed	a random speed	6,186
EventualStop	claims to stop and stay at one position	4,556
Disruptive	content of other vehicles' messages	6,169
DataReplay	replays another vehicle's messages	6,175
DelayedMessages	its own messages, delayed	6,179
DoS	true content at a much higher rate	6,401
DoSRandom	random content at a high rate	6,402
DoSDisruptive	others' content at a high rate	6,407

DoSDisruptive). The four Sybil attacks are not run, and the RandomSpeedOffset archive is excluded because both of its time slices contain the same data. The data are simulated; no claim is made about real road traffic. Table I lists what each attack falsifies and how many attacker slots it contributes to the test set.

What counts as misbehaviour. The benchmark labels vehicles: every message of an attacker vehicle is "malicious". Some attackers, however, send true messages for part of the time; EventualStop, for instance, drives normally until it fakes its stop. Using the per-message ground truth shipped with the dataset, we also score at the message level (Section III-F) and report both.

II. Gaps in Current Work

G1: one sender at a time. Most learned detectors classify a sender, or a pair of consecutive messages, in isolation, from location and movement features [5], [6], [7]. Transformer detectors have appeared [10], [9], including PAMPOS, which flags deviations from a predicted trajectory without attack labels [9]. What a receiver can gain from comparing a sender with the senders around it, and what it loses when those neighbours are themselves malicious, is rarely measured.

G2: metrics. Results are often a single accuracy figure [3]. Accuracy hides the false-positive rate, which decides whether a detector is usable (every false alarm discards an honest vehicle's data), and it says nothing about detection latency.

G3: few attacks, unclear comparisons. Many studies use the original VeReMi with five position attacks [1]. The survey of Abdel Hakeem and Kim [3], which we take as our reference point, lists 97.37% for VeReMi with accuracy, precision, recall and F1 reported together, and per-study accuracies of 95–97% [5], [6]; it does not state splits or label definitions, so such numbers are hard to compare.

G4: consistent lies and collusion. Plausibility checks [4] catch claims that are impossible, but an attacker can fake a trajectory that is internally consistent, and several attackers can agree on one story. Reputation schemes assume most neighbours are honest [8]; learned detectors are rarely tested against coordinated, physically consistent lies.

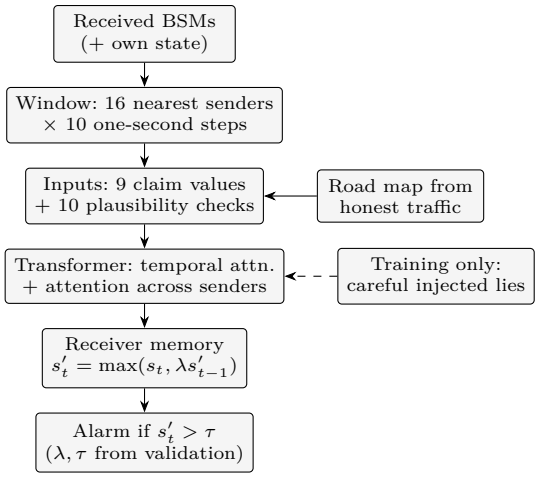


Fig. 1. Detection pipeline on one receiving vehicle.

G5: vehicle-level labels. Scoring every message of an attacker vehicle as malicious counts honest-looking periods as missed detections, which mixes up the detector's errors with the attacker's choice to behave.

G6: where the detector runs. Recent work moves detection into federated learning, to keep vehicle data private [12], [13], or into large language models at the network edge [14]; others target Sybil attacks [15], [16] or combine an LSTM detector with a protocol that reacts to detections [17]. These directions need a strong local detector to start from: one that runs on a single vehicle, is small enough to train and run cheaply, and whose behaviour under attack is measured. That local detector is the focus of this work.

This work. We address G1–G5 with a single receiver-side detector and an evaluation protocol: neighbour attention measured as an ablation and under growing shares of malicious neighbours (G1); precision, recall, F1, accuracy, false-positive rate and latency with significance tests (G2); 14 attack types of VeReMi Extension (G3); plausibility checks and training with careful lies, plus a collusion stress test (G4); and message-level labels next to vehicle-level ones (G5).

III. Our Approach

Fig. 1 shows the pipeline. Raw messages heard by a receiver are grouped into windows, each claim is checked for plausibility, a transformer scores every sender, and a receiver-side memory smooths the scores over time before the alarm threshold is applied.

A. Receiver-centric windows

Each sample is one receiver window as defined in Section I. Per sender and step we keep the raw claim: position, speed, heading, claimed acceleration along the

heading, and the number of messages heard in that second. Sender identities and the dataset's simulated noise fields are never inputs. Windows are skipped when the receiver's own state is missing in any of the 10 steps.

B. Inputs: claims plus plausibility checks

For every sender and step the model receives 19 values. Nine describe the claim itself: position relative to the receiver (scaled by 300 m), speed, cosine and sine of the heading, position change since the previous step, a validity flag, and log(1+messages per second), which makes floods visible. Ten are plausibility checks computed from the claims:

- Kinematics: for claimed position p_t, speed v_t, heading θ_t and acceleration a_t we use a_t, $(v_t - v_{t-1}) - a_t$, $\|p_t - p_{t-1}\| - v_t$ and, when the sender moved more than 1 m, $1 - \cos(\angle(p_t - p_{t-1}) - \theta_t)$. Each is zero for a consistent honest vehicle up to sensor noise.
- Road map: from the receivers' own positions in the training hour we build a 5 m grid of where vehicles actually drive (675×667 cells) with the dominant driving direction per cell. Inputs are the on-road density at the claimed position and the alignment of the claimed heading with the road. The map stands in for the digital map a vehicle carries; no test data are used to build it.
- Neighbours: distance from the claimed position to the nearest other sender's claimed position at the same step.
- Exact repeats: whether the claimed position equals (to 1 cm) one of the sender's own earlier claims, and whether it equals a position that another sender claimed earlier (a copied trail); plus the logarithm of the distance moved, so centimetre-level movement is visible. Real GPS fixes always jitter: no honest step in 1.38 million had zero movement, while frozen and replayed claims repeat exactly.

C. Model

A factorised transformer encodes the window: a linear embedding (d=64) with a learned time encoding, then two blocks, each with (i) temporal self-attention of a sender over its own 10 steps, (ii) spatial self-attention across the senders present at the same step (the neighbourhood), and (iii) a feed-forward layer; 4 heads, dropout 0.1, pre-normalisation. The representation of the newest step feeds a two-layer classification head. Padding and missing steps are masked. The ablation without neighbour attention restricts the spatial attention to the sender itself, so both variants have the same 144k parameters.

D. Training with careful lies

The benchmark's attacks are mostly physically implausible. To teach the model consistent lies as well, we add a copy of half of the training windows (those with at least two honest senders) in which 1–4 honest senders are rewritten as liars, coordinated or independent: a position offset of 20–300 m that keeps the true motion, or a fake stop (position frozen with GPS-like jitter, speed zero, acceleration consistent with the braking). A third lie type, a convoy driving at the receiver, is held out of training to test generalisation. All checks are recomputed on the rewritten claims. This adds 39,691 windows with about 74,000 liar slots.

E. Receiver memory

A receiver that has flagged a sender keeps a decaying suspicion of it: $s'_t = \max(s_t, \lambda s'_{t-1})$ over that receiver's windows of the same sender, forgotten after 30 s of silence. λ and the alarm threshold are chosen on validation data.

F. Message-level labels

A received message is falsified if its claimed position differs from the sender's true position at reception by more than 1 m or its speed by more than 0.5 m/s, using the dataset's ground-truth trace. Honest claims match the ground truth exactly, so the tolerance only absorbs rounding. An attacker slot is positive if the sender sent at least one falsified message in the window; DoS attackers, whose misbehaviour is the flood itself, are always positive. Attacker slots without any falsified message are left out of message-level scoring. They are 2.0% of all attacker slots, almost all from EventualStop before its stop (26% of its slots).

G. Implementation

Everything is written in Python with PyTorch, NumPy and scikit-learn. The VeReMi Extension logs (one JSON file per vehicle, Zenodo record 20090854, CC-BY 4.0) are parsed into windows by a loader that reads message type 2 (the receiver's own state) and type 3 (received BSMs) and takes labels from the file names; 250 receivers are sampled evenly per simulation slice. Parsing runs in parallel and is cached; the features of all windows are written once to half-precision memory-mapped files, so training needs about 1 GB of GPU memory and little RAM. Training uses weighted binary cross entropy (positive weight 2), AdamW (learning rate 2×10^{-3}, weight decay 10^{-2}), a one-cycle schedule, batch size 128 and 8 epochs, about 10–15 minutes per model on a laptop GPU (RTX 3050 Ti). Inference takes well under a millisecond per window. The code, trained models, logs and the scripts that produce every table are in the project repository; the main entry points are run_v2.py (features and training), label_active.py (message labels), run_analysis_v2.py and run_collusion_v2.py. Table II summarises the settings. All random choices (receiver sampling, augmentation, injected lies, bootstrap) use fixed seeds; rebuilding the test windows of our first model from its saved weights on a second machine reproduced its reported numbers exactly.

TABLE II
Implementation and training settings.

Data	VeReMi Extension, 14 attack subsets, 14:00–16:00 traffic
Split	first hour: train; second hour: validation 30% / test 70% of receivers
Windows	109,823 train (+39,691 with injected lies), 29,547 val., 68,971 test
Window	K=16 senders, T=10 steps of 1 s, stride 3 s, 250 receivers per slice
Inputs	19 per sender and step (9 claim values, 10 checks)
Road map	5 m grid, 675×667 cells, receivers' own positions (training hour)
Model	factorised transformer, d=64, 4 heads, 2 blocks, 144k parameters
Training	weighted BCE (pos. weight 2), AdamW 2×10^{-3}, one-cycle, batch 128, 8 epochs
Hardware	laptop, RTX 3050 Ti (4 GB), 16 GB RAM; 10–15 min per model
Software	Python 3.12, PyTorch 2.5, NumPy, scikit-learn
Seeds	2 per model; thresholds and memory decay picked on validation

TABLE III
VeReMi Extension, 14 attack types, 68,971 test windows, mean of 2 seeds (%). Message labels count an attacker as positive only in windows where it sent a falsified message (ground truth); vehicle labels count every window of an attacker vehicle. Thresholds and memory decay are picked on validation. Reference: 97.37% reported for VeReMi (original, 5 attacks) in the survey of Abdel Hakeem and Kim (2025).

Labels	Model	Precision	Recall	F1	Accuracy	FPR
message	Ours	98.65	97.12	97.88	98.69	0.60
message	No nbr. attention	97.99	97.11	97.54	98.48	0.90
message	No lie augm.	98.75	97.39	98.07	98.80	0.56
message	Ours + memory	98.16	97.78	97.97	98.74	–
message	Speed check (rule)	96.21	68.51	80.03	89.34	1.22
vehicle	Ours	98.65	95.41	97.01	98.14	0.60
vehicle	No nbr. attention	97.87	95.48	96.66	97.91	0.96
vehicle	No lie augm.	98.75	95.68	97.19	98.25	0.56
vehicle	Ours + memory	98.16	96.09	97.12	98.20	–
vehicle	Speed check (rule)	95.64	67.21	78.94	88.67	1.41

IV. Evaluation

Protocol. Models are trained on the first simulated hour of each of the 14 attack subsets (109,823 windows) and tested on the second hour, whose receivers are split into contiguous validation (30%, 29,547 windows) and test (70%, 68,971 windows) blocks. Thresholds, memory decay and rule thresholds are chosen on validation only. We report precision, recall, F1, accuracy and false-positive rate (FPR), each the mean of two training seeds (the seeds differ by at most 0.5 points). Per-attack F1 scores an attack against the honest senders of its own simulation run. Differences between models are tested with a paired bootstrap over receivers (2,000 resamples) and Holm correction across attacks. Baseline rules are the range check and the speed-consistency check of VeReMi [1], applied to the same windows.

Overall (Table III). With message labels the detector reaches F1 97.88% (recall 97.12%); with receiver memory F1 is 97.97%, and precision, recall, F1 and accuracy are all above 97.4% in both seeds. With vehicle labels F1 is 97.01% (97.12% with memory): recall falls because the honest-looking windows of attackers count as misses. Our first model, with the same architecture but only the nine raw inputs, reached F1 94.9% (precision 98.4%, recall 91.6%, accuracy 96.9%, vehicle labels); the plausibility checks raised ConstPosOffset recall from 0.66 to 0.97 and EventualStop recall on active windows to 0.99. The speed-consistency rule reaches F1 80.0% and fails completely on offset positions, EventualStop, delayed messages and DoS

TABLE IV
Injected consistent lies (careful: acceleration consistent, GPS-like jitter), m colluders per window, coordinated; colluder recall at a matched 1% validation FPR, mean of 2 seeds. Training saw offset and stop lies only. Before the plausibility checks both models caught at most 4%.

Scenario	Ours	No nbr. attention	No lie augm.
Offset 50 m, $m = 1$	0.85	0.83	0.81
Offset 50 m, $m = 8$	0.83	0.83	0.65
Fake stop, $m = 1$	0.69	0.69	0.01
Fake stop, $m = 8$	0.72	0.70	0.01
Convoy (unseen), $m = 1$	0.87	0.86	0.86
Convoy (unseen), $m = 8$	0.90	0.90	0.72
Offset 50 m along road, $m = 4$	0.25	0.24	0.20
Convoy on own road, $m = 4$	0.42	0.42	0.41

TABLE V
Per-attack results on message labels (mean of 2 seeds). F1 scores each attack against the honest senders of its own simulation run. Δ: F1 gain of neighbour attention.

Attack	Speed check	No nbr. F1	Ours F1	Ours recall	ΔF1
ConstPos	0.869	0.989	0.994	0.999	+0.005
ConstPosOffset	0.023	0.970	0.975	0.965	+0.006
RandomPos	0.967	0.989	0.993	0.998	+0.004
RandomPosOffset	0.966	0.982	0.986	0.984	+0.004
ConstSpeed	0.962	0.985	0.987	0.988	+0.002
ConstSpeedOffset	0.952	0.968	0.970	0.955	+0.002
RandomSpeed	0.966	0.987	0.991	0.993	+0.004
EventualStop	0.000	0.983	0.986	0.991	+0.003
Disruptive	0.968	0.975	0.979	0.970	+0.003
DataReplay	0.946	0.954	0.956	0.927	+0.002
DelayedMessages	0.093	0.920	0.921	0.870	+0.000
DoS	0.023	0.979	0.983	0.978	+0.004
DoSRandom	0.965	0.989	0.994	0.999	+0.004
DoSDisruptive	0.964	0.982	0.985	0.983	+0.003

(Table V). Mean time to the first alarm is 0.9 s (median 0, counted from the first window that contains the sender, which already holds up to 10 s of history); 96% of attacker–receiver pairs are flagged at some point.

Neighbour attention. Attending across senders adds 0.3 F1 points (95% bootstrap interval about [0.2, 0.4] in both seeds and both label schemes), is significantly better on 8–10 of the 14 attacks and significantly worse on none, and lowers the FPR from 0.90% to 0.60%. Its benefit depends on the neighbourhood (Table VI): within each attack type, the gain shrinks as the share of attacker vehicles in the window grows and turns negative (-0.004) once attackers are the majority. In the earlier model without checks the same pattern was stronger (from +0.071 to -0.023). Neighbour context helps while most neighbours are honest; it is not a defence against a malicious majority.

Consistent and colluding lies (Table IV). We inject lies into the test windows, recompute all inputs, and compare models at the same 1% validation FPR. Before the checks, both our models caught at most 4% of such colluders, coordinated or not; neighbour attention even let coordinated convoys raise false alarms on honest neighbours from 0.9% to 3.0%. With the checks, a 50 m offset is caught in 83–86% of cases (63% at 25 m, 98% at 400 m), a fake stop in about 70%, and the held-out convoy in 87–90%. Training with careful lies is what makes fake stops detectable (1% without it) and costs 0.2 F1 points

TABLE VI

Recall on real attackers by the share of attacker vehicles among the senders in the window (message labels, mean of 2 seeds). Last row: recall gain of neighbour attention computed within each attack type (number of attack types with at least 200 attacker slots in the bin).

Attacker share (%)	0–20	20–40	40–60	60–100
Windows	22,310	22,583	15,161	8,917
No neighbour attention recall	0.978	0.974	0.970	0.963
No neighbour attention FPR (%)	0.98	0.93	0.70	1.01
Ours recall	0.979	0.975	0.971	0.959
Ours FPR (%)	0.68	0.64	0.36	0.77
Within-attack gain	+0.002 (4)	+0.001 (14)	+0.001 (14)	-0.004 (14)

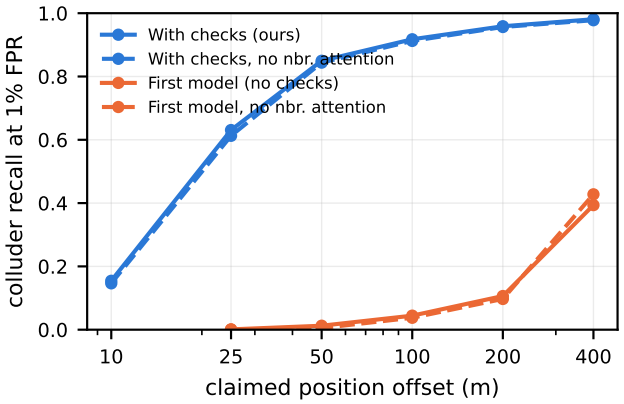


Fig. 2. Recall on four coordinated colluders that shift their claimed position, as a function of the shift, at a matched 1% validation false-positive rate (mean of 2 seeds). Solid: with neighbour attention; dashed: without.

on the benchmark attacks. Fig. 2 shows the effect of the checks as a function of the size of the lie.

Two map-aware attacks remain hard: shifting the claimed position 50 m along the road (23–25%) and a convoy that follows the receiver's own road (36–48%).

Reference numbers. The survey [3] lists 97.37% for VeReMi (accuracy, precision, recall and F1 reported together) and F1 98% for positional attacks with a reputation-based detector [8]; So et al. report precision 95.91% and detection rate 83.73% with physical-layer checks [4]. Our message-level results exceed 97.37% on all four measures (position attacks alone: F1 97.5–99.4%). These figures come from a different dataset (original VeReMi, five attacks) and partly undocumented protocols; they place our results but are not a like-for-like comparison.

V. Discussion

What the detector relies on. The progression from our first model to the final one shows where the accuracy comes from. The first model saw the same senders and the same neighbourhood but only the raw claims; it learned to flag claims that are physically impossible (positions beyond radio range, speed that does not match movement) and missed everything else, including 34% of ConstPosOffset attackers. The plausibility checks turn knowledge a vehicle already has (a map, the laws of motion, the fact that GPS readings never repeat exactly) into explicit inputs, and the largest gains come from exactly the attacks that are consistent in themselves: offsets inside the radio range, frozen positions and replayed trails. Neighbour attention is a second-order effect on top: it lowers false alarms, probably because an honest vehicle that looks unusual (for example, one stopping hard) is easier to accept when its neighbours behave the same way.

Neighbours as evidence, and as a risk. The attacker-share analysis and the stress test agree. Comparing a sender with its neighbours is useful when the neighbours are mostly honest, and it becomes a liability when they are not: coordinated liars barely affect their own detection but can make the model doubt honest vehicles around them. A deployed system should therefore not treat agreement among neighbours as proof, and it should cap how much any group of senders can change the score of another sender.

Deployment. The model has 144k parameters and scores a window in well under a millisecond on a laptop GPU, so a single vehicle could run it on every incoming window. Three parts need care in practice. The road map must come from map data rather than from recorded traffic; the alarm threshold trades false alarms against missed attackers and should be set per deployment; and the receiver memory delays the release of a falsely flagged sender by a few seconds, which is acceptable for misbehaviour reports but not for discarding safety-critical messages outright.

An adaptive attacker. An attacker who knows the checks will keep its lies on the road, smooth and consistent with its claimed speed and acceleration, and will add jitter to frozen positions. Our map-aware stress test models this attacker, and detection drops to 23–48%. Content checks alone cannot close this gap: the receiver would need independent evidence of where the sender really is, such as signal strength or angle of arrival, its own camera and radar, or reports from roadside units.

VI. Limitations and Conclusion

The benchmark is simulated; 14 of 19 attack types are covered (no Sybil attacks); results are the mean of two seeds; dense rush-hour traffic was evaluated only for the earlier model; the road map assumes the receiver has a map of the area; and the collusion test rewrites honest vehicles of the trace instead of simulating new attackers, without re-selecting the 16 senders after the lie. The message-level labels are a choice we make explicit, and we report vehicle-level results next to them.

Within these limits, the study gives three answers. Plausibility checks computed from the claims (kinematics, a learned road map, exact repeats and copied trails) did most of the work: they moved the detector from F1 94.9% to about 98% and turned consistent lies from invisible to mostly detected. Neighbour attention gives a small,

significant gain and fewer false alarms, but its benefit disappears once most neighbours are malicious. A lie that stays on the road and obeys the physics cannot be separated from the truth using message content alone; closing that gap needs an independent signal such as received signal strength, on-board sensors or roadside units, together with Sybil attacks and the newer VeReMi NextGen data, which are our next steps.

References

[1] R. W. van der Heijden, T. Lukaseder, and F. Kargl, "VeReMi: A dataset for comparable evaluation of misbehavior detection in VANETs," in Security and Privacy in Communication Networks (SecureComm). Cham, Switzerland: Springer, 2018, pp. 318–337.

[2] J. Kamel, M. Wolf, R. W. van der Heijden, A. Kaiser, P. Urien, and F. Kargl, "VeReMi Extension: A dataset for comparable evaluation of misbehavior detection in VANETs," in Proc. IEEE Int. Conf. Commun. (ICC), Jun. 2020, doi: 10.1109/ICC40277.2020.9149132.

[3] S. A. Abdel Hakeem and H. Kim, "Advancing intrusion detection in V2X networks: A comprehensive survey on machine learning, federated learning, and edge AI for V2X security," IEEE Trans. Intell. Transp. Syst., vol. 26, no. 8, pp. 11137–11205, Aug. 2025.

[4] S. So, J. Petit, and D. Starobinski, "Physical layer plausibility checks for misbehavior detection in V2X networks," in Proc. 12th Conf. Security and Privacy in Wireless and Mobile Networks (WiSec), 2019, pp. 84–93.

[5] S. So, P. Sharma, and J. Petit, "Integrating plausibility checks and machine learning for misbehavior detection in VANET," in Proc. 17th IEEE Int. Conf. Machine Learning and Applications (ICMLA), 2018, pp. 564–571.

[6] P. K. Singh, S. Gupta, R. Vashistha, S. K. Nandi, and S. Nandi, "Machine learning-based approach to detect position falsification attack in VANETs," in Security and Privacy. Singapore: Springer, 2019, pp. 166–178.

[7] A. Sharma and A. Jaekel, "Machine learning based misbehaviour detection in VANET using consecutive BSM approach," IEEE Open J. Veh. Technol., vol. 3, pp. 1–14, 2022.

[8] S. Gyawali, Y. Qian, and R. Q. Hu, "Machine learning and reputation based misbehavior detection in vehicular communication networks," IEEE Trans. Veh. Technol., vol. 69, no. 8, pp. 8871–8885, Aug. 2020.

[9] K. Kalogiannis, A. M. Hussain, and P. Papadimitratos, "PAMPOS: Causal transformer-based trajectory prediction for attack-agnostic misbehavior detection in V2X networks," arXiv:2605.06833, 2026.

[10] H. Li, K. Kalogiannis, A. M. Hussain, and P. Papadimitratos, "AttentionGuard: Transformer-based misbehavior detection for secure vehicular platoons," arXiv:2505.10273, 2025.

[11] S. Ruj, M. A. Cavenaghi, Z. Huang, A. Nayak, and I. Stojmenovic, "Data-centric misbehavior detection in VANETs," arXiv:1103.2404, 2011.

[12] E. Mármol Campos, A. González Vidal, J. L. Hernández Ramos, and A. Skarmeta, "Federated learning for misbehaviour detection with variational autoencoders and Gaussian mixture models," arXiv:2405.09903, 2024.

[13] U. Demir, Y. E. Sagduyu, T. Erpek, H. Jafari, S. Kompella, and M. Xue, "Distributed federated learning for vehicular network security: Anomaly detection benefits and multi-domain attack threats," arXiv:2505.23706, 2025.

[14] W. Hamhoum and S. Cherkaoui, "MistralBSM: Leveraging Mistral-7B for vehicular networks misbehavior detection," arXiv:2407.18462, 2024.

[15] M. Thomas, A. Borah, and A. Paranjothi, "Trust-aware Sybil attack detection for resilient vehicular communication," Internet Technology Letters, 2024, arXiv:2411.07520.

[16] M. S. T. Khan, M. S. A. Utsho, and M. Jahan, "A robust framework for Sybil attack detection in vehicular ad hoc networks," arXiv:2606.11667, 2026.

[17] M. Franceschini, "A neural-network based anomaly detection system and a safety protocol to protect vehicular network," Master's thesis, arXiv:2411.07013, 2024.

[18] A. Boualouache and T. Engel, "A survey on machine learning-based misbehavior detection systems for 5G and beyond vehicular networks," arXiv:2201.10500, 2022.